Without styling, it is rather jumbled, but your users can now complain about that with your new comment form.

Test your form out by leaving your name and comment. As you're using the `console.EmailBackend`, the resulting email will display in the terminal.

7.1.2 CSRF

You've been patient with my hand-waving and references to black magic, so just what is the `{% csrf_token %}`? *Cross-site request forgery* (CSRF) is an attack that tricks a user into doing something on your site that they didn't intend. A malicious site can be designed to look exactly like your site and use a form that looks like an innocuous task but actually causes a change on the real site. For example, the malicious site might have a form that looks just like your comment form but contains hidden fields for doing a password change. When the user hits Submit, the malicious form communicates with your real site, causing the user's password to change. The bad hats can then use the changed password to access the user's account.

To prevent this attack, Django uses the `{% csrf_token %}`. It works through a piece of Django Middleware called `CsrfViewMiddleware`, which creates a session cookie containing a random value. A variation on this value gets included as a hidden form field when you use `{% csrf_token %}`. When your form gets submitted, the value gets included in the POST. Django looks for this token, and if it isn't there or doesn't match the one in the session, the POST gets rejected.

The malicious site has no way of knowing the value of the CSRF token, so a fake form will not be able to include it. For more information on CSRF attacks in general, see the Open Worldwide Application Security Project's (OWASP) "Cross Site Request Forgery" article (https://owasp.org/www-community/attacks/csrf). For specifics on how Django prevents the attack, see the CSRF documentation: https://docs.djangoproject.com/en/dev/ref/csrf/.

7.1.3 Beautifying web forms

I haven't seen a web page without CSS since the 1990s. It sounds like a bad joke: the author's so old he used the web before it was stylized! The form examples in this and previous chapters have all used `form.as_p` to render the necessary HTML tags. In addition to `form.as_p`, there is `form.as_table`, but this doesn't do much to make the results prettier. Table columns mean the labels in the forms all end up the same width, which is an improvement, but it still won't win any design awards.

Earlier, you saw how you could use the `attrs` argument to a widget to add attributes to the resulting HTML tag. The example populated the `rows` and `cols` attributes of a `<textarea>` tag. This mechanism can also be used to add a `class` attribute to the tag, making it easier to stylize the result with CSS.

Class attributes may not be enough, depending on what you are trying to achieve. For example, Bootstrap has some very pretty forms, but to get them, you need to wrap the tags in other tags. At this point, you're beyond the limit of what the Django forms